EXPERT ELECTRONICS

Public Documentations

TCI Protocol

Ver. 2.1

Series: Expert Electronics' products external integration capabilities and APIs

September 2026

Proprietary Notice

Confidentiality

All information contained in this document is for open public use. It is provided for the purpose of technical information for 3rd party vendors or individuals, who wish to integrate their software/hardware with Expert Electronics' products.

Obligations and Conditions

Copyright (c) 2023 "Expert Group" LLC, Taganrog

Permission is hereby granted, free of charge, to any person obtaining a copy of this software (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Point of contact

Name:	"Expert Group" LLC		
Tel:	+7 (8634) 43-13-01	Email:	info@sunsdr.com
Address:	347927, Russia, Rostov region, Taganrog, Polyakovskoe schosse, 16-3, ABK-1, off. 408		
Website:	https://eesdr.com/en/		

Document Control

1.9	New Commands: AUDIO_STREAM_SAMPLE_TYPE AUDIO_STREAM_CHANNELS AUDIO_STREAM_SAMPLES DIGL_OFFSET DIGU_OFFSET TX_STREAM_AUDIO_BUFFERING Other changes: Several important updates in the "Working with audio streams via TCI" section 3.4.	29 July 2022	0.13.0
1.9.1	New commands: • KEYER	31 May 2023	1.0.4
2.0	Changes in commands: • VFO_LOCK • RX_CHANNEL_SENSORS Other changes: • Updated TRX command	12 Jan 2024	1.0.7
	New commands: RX_ENABLE ECODER_SWITCH_RX ECODER_SWITCH_CHANNEL HW_SENSORS_ENABLE HW_SENSORS CW_MON Renamed: CHANNEL_COUNT → CHANNELS_COUNT First described as tables: CW_MACROS CW_MSG CW_MACROS_STOP CW_TERMINAL		

2.1	CW_MACROS_EMPTY CALLSIGN_SEND Changes in commands: KEYER TX_ENABLE RX_CHANNEL_ENABLE TRX CW_KEYER_SPEED IQ_SAMPLERATE AUDIO_SAMPLERATE AUDIO_STREAM_SAMPLE_TYPE AUDIO_STREAM_CHANNELS AUDIO_STREAM_SAMPLES TX_STREAM_AUDIO_BUFFERING RX_SENSORS Other changes: New sections: Reserved Commands and Notes on the Command Format; count field added to the Stream structure	September 2026	1.3.1

Contents

1 Introduction 4
1.1 Document Scope 4
1.2 Document Purpose 4
1.3 Document Audience 4
1.4 Context 4
2 Updates and Changes in this version 5
2.1 Updated commands 5
2.2 New Commands 5
2.3 Other changes 5
3 General Overview 6
3.1 Interface Description 6
3.2 Working in CW mode 6
3.2.1 CW macro 7
3.2.2 CW message 8
3.3 Types of TCI commands 9
3.4 Working with audio streams via TCI 9
3.5 Particularities of TCI server operation in ExpertSDR3 11
3.6 Conclusion 11
4 TCI Commands List 12
4.1 Initialization Commands 12
4.2 Bidirectional Control Commands 14
4.3 Unidirectional Control Commands 29
4.4 Notification Commands 36
4.5 New Commands in this version 40
Appendix A. Glossary of Technical Terms 41

1 Introduction

1.1 Document Scope

TCI (Transceiver Control Interface) is a network interface for control, data transfer and synchronization between transceiver/receiver, contest loggers, digital mode software, skimmers and other software, as well as external power amplifiers, bandpass filter units, antenna switches, radio controllers and other devices.

This document is a living document and will be updated and validated every time when TCI protocol is updated.

1.2 Document Purpose

This document describes the TCI protocol, what it's for and how to use it.

1.3 Document Audience

The target audience of this document is programmers who implement TCI protocol in their programs and devices.

1.4 Context

TCI was created as a modern alternative to the outdated COM port and audio cable interfaces, it uses a full duplex web socket protocol that runs on top of a TCP connection and serves for server-client communications, providing cross-platform connectivity. Transceiver works as a server, all other software and devices as clients. The server and clients can be inside the same computer (program-server, hardware log, etc.-clients) and/or in separate physical devices connected through the local network (classical transceiver, power amplifier, antenna switch, FFT unit, etc.).

The TCI interface contains basic transceiver control commands (similar to CAT system), receives CW macros from clients and broadcasts them, outputs transceiver IQ stream to clients, receives spots from skimmers and Internet clusters, receives/outputs audio signal to work in digital modes.

The TCI uses an extensible architecture and can be supplemented with new functions and commands, while keeping the old ones operational. Thus, the TCI interface can be extended and supplemented to meet the specific needs of any software manufacturer and/or device manufacturer (receivers, transceivers, power amplifiers, switches, etc.). The presence of a device identifier allows the manufacturers of transceivers and receivers to switch to the TCI interface while maintaining the device model designation. The extensibility of the TCI interface allows you to create an individual set of commands and functions for each device model, while maintaining the basic command set inherent to all transceivers.

Expert Electronics advocates universal unification of data exchange between devices and software by creating the TCI interface for this purpose. Modern transceivers and software must communicate using one protocol - the TCI protocol.

2 Updates and Changes in this version

2.1 Updated commands

KEYER

TX_ENABLE

RX_CHANNEL_ENABLE

CW_KEYER_SPEED

IQ_SAMPLERATE

AUDIO_SAMPLERATE

AUDIO_STREAM_SAMPLE_TYPE

AUDIO_STREAM_CHANNELS

AUDIO_STREAM_SAMPLES

TX_STREAM_AUDIO_BUFFERING

RX_SENSORS

- TRX

2.2 New Commands

RX_ENABLE

ECODER_SWITCH_RX

ECODER_SWITCH_CHANNEL

HW_SENSORS_ENABLE

HW_SENSORS

CW_MON

Renamed: CHANNEL_COUNT → CHANNELS_COUNT

CW commands first described as tables: CW_MACROS, CW_MSG, CW_MACROS_STOP, CW_TERMINAL, CW_MACROS_EMPTY, CALLSIGN_SEND

2.3 Other changes

New sections: Reserved Commands and Notes on the Command Format

count field added to the Stream structure (reserv[8] → reserv[7])

3 General Overview

3.1 Interface Description

Any command represents an ASCII string that contains a command name and a list of arguments corresponding to this command. There are reserved characters that cannot be included in the command name and command arguments.

List of reserved characters: «:», «,», «;».

Command structure:
1. Name of the command;
2. Separating character between command name and arguments «:»;
3. Separating character between arguments «,»;
4. End of the command character «;».

If a command has no arguments, an end of command symbol is placed after the command name. If the command is invalid, it is ignored. The case of letters does not matter.

The ExpertSDR3 program acts as a server, which can have several client connections at the same time, they will be synchronized with each other by the server. When connecting to the ExpertSDR3, the client receives the current status of the ExpertSDR3, first sending initialization commands, then parameters to set the status, such as frequency, modulation, etc.

When a parameter change occurs in the ExpertSDR3 (server) program, the server notifies all connected clients, i.e., clients do not need to poll the server constantly, any change of state will be sent in time to all clients. If the client sends a new state, the server will set it to itself, as well as send it to all clients, that is, the server acts as a synchronizer. All clients connected to the server will be automatically synchronized. This way of work allows to minimize network load, reducing traffic.

The TCI protocol implements the transmission of receiver IQ stream to clients, which is necessary for the work of special skimmer software, they automatically find the station and decode it throughout the band, and it also allows you to record radio signals in the file.

TCI is also used to transmit audio signals of the receiver to clients and to receive audio signals from clients, i.e., the client can transmit audio signals to ExpertSDR3 for radio transmission. The audio stream exchange is designed to work with digital modes, where encoding and decoding is performed by third-party software, as well as in voice modes, where audio macros can be broadcasted, which is very much in demand in contest loggers.

When working in contests, it is important to record all on the air operation, for this purpose the audio stream from the line-output is sent to all clients. The resulting audio stream can be recorded to a file or played back with a PC sound card.

3.2 Working in CW mode

Through the TCI protocol CW can be generated from string commands. The commands are divided into two types:

1. CW Macros;
2. CW Message.

3.2.1 CW macro

A CW macro is a set of characters that has no rules, but still can include commands to change the CW speed and or to send CW abbreviations. CW macros can be used in CW terminal mode.

The command to send a macro has the form: cw_macros:arg1,arg2;

arg1 - is the periodic number of the software transceiver;

arg2 - text message.

Example: cw_macros:0,TU RA6LH 599;

To put a CW abbreviation into the text (to combine several letters together), you should put the characters between the vertical brackets, for example:

TEXT |SK| TEXT.

To change the CW speed within a text, "<" decreases the speed, and ">" increases the speed respectively. The speed step is 5 wpm, for example:

ANY TEXT > TEXT+5WPM >>TEXT+15WPM

To pass the string «+5wpmTU -5wpmRA6LH +10wpm599 004 *SK*» the command would have the form:

cw_macros:0,>TU <RA6LH >>599 004 |SK|;

As text commands may contain characters which are reserved by the protocol, they are replaced by other characters and converted back in the TCI server:

1. The character «:» is replaced with «^»;
2. The character «,» is replaced with «~»;
3. The character «;» is replaced with «*».

When transmitting macros, it is possible to operate in terminal mode, i.e. the transceiver remains in transmission after the macro transmission is completed until terminal mode is disabled. The following command enables/disables the terminal mode:

- cw_terminal:true; - enable;
- cw_terminal:false; - disable.

In terminal mode, a command is sent to the client as soon as the last letter in the queue start to be transmitted: cw_macros_empty;

If the terminal mode is disabled during macro playback, the transceiver will automatically switch to receive mode when the macro is finished.

3.2.2 CW message

A CW message is a special command that has a more complex structure and consists of three parts:

1. The prefix is the text before the callsign;
2. The callsign;
3. The suffix is the text after the callsign.

The command sends a CW message and offers the possibility of repeating the callsign:
cw_msg:arg1,arg2,arg3,arg4;

- arg1 - is the number of the software transceiver
- arg2 - prefix
- arg3 - callsign
- arg4 - suffix

To pass the string «TU RA6LH 599 004» the command would have the form:

cw_msg:0,TU,RA6LH,599 004;

The parts are joined with a single space, so the client does not have to add spaces itself; a space already put by the client at a joint is not doubled. The prefix «_» means an empty prefix.

If the callsign is to be repeated twice «TU RA6LH RA6LH 599 004», the command will have the form:

cw_msg:0,TU,RA6LH$2,599 004;

When a CW message is already sent to TCI but the callsign is not sent yet, the callsign can be corrected using the command:

cw_msg:arg1;

An example of a callsign correction sequence:
1. cw_msg:0,_,RA6$2,599 004;
2. cw_msg:RA6L;
3. cw_msg:RA6LH;

If editing of the callsign was performed after the callsign transmission was completed, the callsign correction command is ignored. The process of editing the callsign is performed for characters that have not yet been transmitted. After the transmission of the callsign is

completed, a command is sent by the TCI server to the client, containing the final version of the callsign transmitted:

callsign_send:ra6lh;

The transmission of CW macros or messages can be immediately stopped with the command:

cw_macros_stop;

3.3 Types of TCI commands

Several types of commands are provided in the protocol:

- Initialization commands;
- Bi-directional control commands;
- Unidirectional control commands;
- Notification Commands.

Initialization commands are sent to the client upon connection, they inform the client of the basic device parameters such as operating frequency range, supported modes, etc.

Bi-directional control commands are needed to control the basic parameters of the device and ExpertSDR3. Since the commands are bidirectional, the server will synchronize the parameters of all connected clients.

Unidirectional control commands are used to control parameters unique to each client, such as enabling audio streaming, sending spots to be displayed on an ExpertSDR3 panorama, etc.

Notification commands are sent to clients at some intervals, these can be readings from various sensors and signal meters.

3.4 Working with audio streams via TCI

Thanks to the WebSocket protocol, commands and audio streams are separated, commands are transmitted as strings and audio streams are transmitted as byte streams. Audio streams are transmitted in blocks of bytes, a block has a header and a data field.

Structure of a block in C language:

```
struct Stream
{
uint32_t receiver;          // receiver number
uint32_t sample_rate;       // sampling rate
uint32_t format;            // sample type determined in SampleType
uint32_t codec;             // compression algorithm (not implemented), always 0
uint32_t crc;               // checksum (not implemented), always 0
uint32_t length;            // number of samples
uint32_t type;              // stream type
uint32_t channels;          // number of channels
uint32_t count;             // packet counter
uint32_t reserv[7];         // reserved
uint8_t data[16384];        // samples
```

};

The count field is a packet counter incremented for each packet within a stream. It occupies the first of the words reserved in TCI 1.8, so the header layout remains binary compatible. The counter is kept separately for the RX audio stream and the TX_CHRONO stream and is reset when switching between TX and RX. The header is still 64 bytes.

The flow type is defined by an enumeration:

```
enum StreamType
{
IQ_STREAM = 0,              // Receiver IQ signal stream
RX_AUDIO_STREAM = 1,        // Receiver audio stream
TX_AUDIO_STREAM = 2,        // Audio stream for transmitter
TX_CHRONO = 3,              // Time marker flow for audio signal transmission
LINEOUT_STREAM = 4          // Linear audio output audio stream
}
enum SampleType
{
INT16 = 0,                  // 16-bit integral character type
INT24 = 1,                  // 24-bit integral character type
INT32 = 2,                  // 32-bit integral character type
FLOAT32 = 3,  // 32-bit type with a floating point
}
```

After connection, the client can enable receiving the IQ stream with the IQ_START command, after sending this command ExpertSDR3 starts sending the IQ stream. In the Stream.length field specified the number of real samples indicated in the uint8_t data[] field, since the these are complex samples, so the number of complex samples should be calculated as Stream.length/Stream.channels.

The receiver audio stream completely duplicates the IQ stream, but the difference is that you can control certain parameters, such as:

- Channels number
- Samples format
- Number of samples sent in a single packet

When DIGL/DIGU modes are selected, a complex signal will be transmitted if the number of channels is 2, but if the number of channels is 1 then it's going to be a real signal. Audio stream of the Line Out duplicates the regular audio stream, but you cannot change its parameters.

Sending an audio stream to a transmitter has certain peculiarities: ExpertSDR3 sends a TX_CHRONO timestamp that notifies the client to send an audio signal marked as TX_AUDIO_STREAM with the specified number of samples in the Stream.length. Timestamps are sent without waiting for a response from the client. If the signal is not ready to be sent to

ExpertSDR3, the client may not send a response or may send a signal with zero counts, which corresponds to no signal - this option is preferable.
There are changes in the Stream structure in comparison to the version 1.8, but binary compatibility is consistent, all programs which support TCI 1.8 will work with TCI 2.1 as well.

3.5 Particularities of TCI server operation in ExpertSDR3

Server syncs all the clients connected to it – this is a difficult task, that is why there are certain particularities in TCI server operation, which you need to consider developing your client software.

There are a lot of commands and the list grows with each new ExpertSDR3 version, but it doesn't prevent effective sync process between clients. The correct hierarchy is the basics, which definitely defines master and slave software. If several clients will try to change the same parameter at the same time, the priority is behind the first one, but the highest priority is behind the ExpertSDR3. Even if a certain client controls some parameter, ExpertSDR3 operator will always be able to take control of this.

It's important to know, that in ExpertSDR3 we've implemented saving of certain parameters by band and mode. When a band has been changed all clients get new parameters in a certain sequence. For example, if a client sets frequency, which is different from the current band, ExpertSDR3 will set this frequency and change the band, then ExpertSDR3 will reinstate the settings saved for this band and send them to connected clients. When ExpertSDR3 initiates a change of any parameter, it monopolizes it for 200 ms and clients can't change it for this time period. The same rule is applied if one of the clients controls a parameter. At this moment other clients can only get notified about the change, but cannot control it until 200 ms will pass after the moment the first client stopped changing this parameter.

CW macros operation also has its own peculiarities, since it was designed especially for contests, but of course you can use them in your everyday operation. There are two commands for sending a text message via CW in TCI. The first and simplest is the CW_MACROS command, it just sends a text on the air. During the transmission of the CW macros, you can add more macros in line so all of them will be transmitted on the air in their turn. The second command is CW_MSG, it has a more complicated structure, prefix, callsign and suffix. Until the callsign is fully transmitted on the air it can be edited, but if during the transmission of such message you'll send another CW_MSG with prefix, callsign and suffix, then the current transmission will stop and the new one will start immediately. If during the transmission of the CW_MSG you'll send CW_MACROS it will be added in line after the active transmission. If during the transmission of the CW_MACROS you'll send CW_MSG, it immediately stops the transmission of CW macros and starts transmission of message, because message has a higher priority.

3.6 Conclusion

The TCI protocol is constantly evolving. With the release of each new version of ExpertSDR3, the number of commands expands to increase the functionality of the programs and devices connected to ExpertSDR3.

The development team always listens to the opinion of ExpertSDR3 users. We try to make our product user-friendly both for software developers, allowing them to spend less time on

integration of their programs and devices, and for common users, who are interested not in technical aspects but in convenience and ease of use.

Each of you can contribute to the development of the program. Your suggestions, wishes and comments will help us to make ExpertSDR3 better.

4 TCI Commands List

4.1 Initialization Commands

VFO_LIMITS	Device operating frequency range.	
Reply	VFO_LIMITS:arg1,arg2;	arg1 — lower frequency limit, Hz. arg2 — top frequency limit, Hz.
Type	Initialization	
Example	VFO_LIMITS:10000,30000000;	

IF_LIMITS	Frequency limit values of the IF filter (for VFOA only).	
Reply	IF_LIMITS:arg1,arg2;	arg1 — lower frequency limit, Hz. arg2 — top frequency limit, Hz.
Type	Initialization	
Example	IF_LIMITS:-48000,48000;	
Note	Sent when you connect a device or change the sample rate.	

TRX_COUNT	Number of receivers (transceivers) in the radio.	
Reply	TRX_COUNT:arg1;	arg1 — number of receivers/transceivers (physical or software).
Type	Initialization	
Example	TRX_COUNT:2;	

CHANNELS_COUNT	Number of receiving channels in one receiver (A/B/C).	
Reply	CHANNELS_COUNT:arg1;	arg1 — number of receive channels.
Type	Initialization	
Example	CHANNELS_COUNT:2;	
Note	Until TCI 2.1 the command was named CHANNEL_COUNT (singular). The old name	

	is no longer sent.

DEVICE	Device Name.	
Reply	DEVICE:arg1;	arg1 — device name.
Type	Initialization	
Example	DEVICE:SunSDR2DX;	

RECEIVE_ONLY	Identifies the device as a receiver or transceiver.	
Reply	RECEIVE_ONLY:arg1;	arg1 — receiver only (true), transceiver (false).
Type	Initialization	
Example	RECEIVE_ONLY:true;	

MODULATIONS_LIST	List of supported modulations.	
Reply	MODULATIONS_LIST:arg1,arg2, … ,argN;	The modulation is conveyed by the name.
Type	Initialization	
Example	MODULATIONS_LIST:AM,LSB,USB,FM;	

PROTOCOL	TCI protocol version.	
Reply	PROTOCOL:arg1,arg2;	arg1 — program name. arg2 — version of the protocol.
Type	Initialization	
Example	PROTOCOL:ExpertSDR3,2.1;	

READY	Sent after the initialization commands.

Reply	READY;	
Type	Initialization	

4.2 Bidirectional Control Commands

START	Device start	
Set	START;	
Type	Bidirectional control	
Example	START;	

STOP	Device stop	
Set	STOP;	
Type	Bidirectional control	
Example	STOP;	

DDS	Receiver center frequency control	
Set	DDS:arg1,arg2;	arg1 — receiver periodic number; arg2 — tuning frequency, Hz.
Read	DDS:arg1;	
Reply	DDS:arg1,arg2;	
Type	Bidirectional control	
Example	DDS:0; DDS:0,7100000;	

IF	Frequency control of the IF tuning filter within a panorama	
Set	IF:arg1,arg2,arg3;	arg1 — receiver periodic number; arg2 — channel number (A / B); arg3 — tuning frequency, Hz.

Read	IF:arg1,arg2;	
Reply	IF:arg1,arg2,arg3;	
Type	Bidirectional control	
Example	IF:0,1; IF:0,1,12500; IF:0,1,-17550;	

VFO	Receiver frequency control	
Set	VFO:arg1,arg2,arg3;	arg1 — receiver periodic number; arg2 — channel number (A/B); arg3 — tuning frequency, Hz.
Read	VFO:arg1,arg2;	
Reply	VFO:arg1,arg2,arg3;	
Type	Bidirectional control	
Example	VFO:0,1,7100000; VFO:1,0,14250000; VFO:0,1;	

MODULATION	Switching modes	
Set	MODULATION:arg1,arg2;	arg1 — receiver periodic number; arg2 — mode (string).
Read	MODULATION:arg1;	
Reply	MODULATION:arg1,arg2;	
Type	Bidirectional control	
Example	MODULATION:0,LSB; MODULATION:1; MODULATION:1,NFM;	

Note	A list of supported modes is sent to the client when connecting to ExpertSDR3.

TRX	Switching RX/TX modes	
Set	TRX:arg1,arg2,arg3;	arg1 — transceiver periodic number; arg2 — status indicator (enable – true, disable – false). arg3 — signal source (optional) tci - take signal from TCI audio stream. mic1 - take signal from Mic1 mic2 - take signal from Mic2 micPC - take signal from MicPC ecoder2 – take signal from E-Coder2 If an argument is absent, the signal is taken from the microphone.
Read	TRX:arg1;	
Reply	TRX:arg1,arg2;	
Type	Bidirectional control	
Example	TRX:0,true; TRX:0,true,tci; TRX:0,false; TRX:0,true,micpc; TRX:0,true,ecoder2; TRX:0,true,mic2; TRX:1;	
Note	The signal for transmitting is always taken from the microphone selected in the ExpertSDR3. If a third-party software connected via TCI wants to transmit its audio signal, you must specify the third argument - TCI. This works if the TCI audio stream is enabled, otherwise the microphone signal is taken. If you must specify a certain input for the transmition, select one of the mic inputs. If TCI is not specified, then enabling the TCI audio stream is not required.	

TUNE	Switching between receive and Tune modes	
Set	TUNE:arg1,arg2;	arg1 — transceiver periodic number; arg2 — status indicator (enable – true, disable – false).
Read	TUNE:arg1;	
Reply	TUNE:arg1,arg2;	

Type	Bidirectional control	
Example	TUNE:0,true; TUNE:0,false; TUNE:1;	

DRIVE	Output power control	
Set	DRIVE:arg1,arg2;	arg1 — transceiver periodic number; arg2 — output power value from 0 to 100.
Read	DRIVE:arg1;	
Reply	DRIVE:arg1,arg2;	
Type	Bidirectional control	
Example	DRIVE:0,30; DRIVE:0,75; DRIVE:1;	

TUNE_DRIVE	Output power control in Tune mode	
Set	TUNE_DRIVE:arg1,arg2;	arg1 — transceiver periodic number; arg2 — output power value from 0 to 100.
Read	TUNE_DRIVE:arg1;	
Reply	TUNE_DRIVE:arg1,arg2;	
Type	Bidirectional control	
Example	TUNE_DRIVE:0,30; TUNE_DRIVE:0,75; TUNE_DRIVE:1;	

RIT_ENABLE	Enable RIT	
Set	RIT_ENABLE:arg1,arg2;	arg1 — receiver periodic number; arg2 — status indicator (enable – true, disable – false).

Read	RIT_ENABLE:arg1;	
Reply	RIT_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	RIT_ENABLE:0,true; RIT_ENABLE:1;	

XIT_ENABLE	Enable XIT	
Set	XIT_ENABLE:arg1,arg2;	arg1 — transceiver periodic number; arg2 — status indicator (enable – true, disable – false).
Read	XIT_ENABLE:arg1;	
Reply	XIT_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	XIT_ENABLE:0,true; XIT_ENABLE:1;	

SPLIT_ENABLE	Enable Split operation	
Set	SPLIT_ENABLE:arg1,arg2;	arg1 — transceiver periodic number; arg2 — status indicator (enable – true, disable – false).
Read	SPLIT_ENABLE:arg1;	
Reply	SPLIT_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	SPLIT_ENABLE:0,true; SPLIT_ENABLE:1;	

RIT_OFFSET	Adjust RIT offset

Set	RIT_OFFSET:arg1,arg2;	arg1 — receiver periodic number; arg2 — offset frequency, Hz.
Read	RIT_OFFSET:arg1;	
Reply	RIT_OFFSET:arg1,arg2;	
Type	Bidirectional control	
Example	RIT_OFFSET:0,500; RIT_OFFSET:1;	

XIT_OFFSET	Adjust XIT offset	
Set	XIT_OFFSET:arg1,arg2;	arg1 — transceiver periodic number; arg2 — offset frequency, Hz.
Read	XIT_OFFSET:arg1;	
Reply	XIT_OFFSET:arg1,arg2;	
Type	Bidirectional control	
Example	XIT_OFFSET:0,-350; XIT_OFFSET:1;	

RX_CHANNEL_ENABLE	Enable additional receive channel (VFO B)	
Set	RX_CHANNEL_ENABLE:arg1,arg2,arg3;	arg1 — receiver periodic number; arg2 — channel periodic number. arg3 — status indicator (enable – true, disable – false).
Read	RX_CHANNEL_ENABLE:arg1,arg2;	
Reply	RX_CHANNEL_ENABLE:arg1,arg2,arg3;	
Type	Bidirectional control	
Example	RX_CHANNEL_ENABLE:0,1,true; RX_CHANNEL_ENABLE:0,1;	

RX_FILTER_BAND	Adjust RX filter width	
Set	RX_FILTER_BAND:arg1,arg2,arg3;	arg1 — receiver periodic number. arg2 — lower frequency limit, Hz. arg3 — top frequency limit, Hz.
Read	RX_FILTER_BAND:arg1;	
Reply	RX_FILTER_BAND:arg1,arg2,arg3;	
Type	Bidirectional control	
Example	RX_FILTER_BAND:0,30,2700; RX_FILTER_BAND:1,-2900,-70; RX_FILTER_BAND:0;	

CW_MACROS_SPEED	Control CW macros speed (WPM)	
Set	CW_MACROS_SPEED:arg1;	arg1 — CW speed, WPM (1…60).
Read	CW_MACROS_SPEED;	
Reply	CW_MACROS_SPEED:arg1;	
Type	Bidirectional control	
Example	CW_MACROS_SPEED:42; CW_MACROS_SPEED;	

CW_MACROS_DELAY	CW macros TX delay	
Set	CW_MACROS_DELAY:arg1;	arg1 — delay before the start of CW transmission, ms (10…1000).
Read	CW_MACROS_DELAY;	
Reply	CW_MACROS_DELAY:arg1;	
Type	Bidirectional control	
	CW_MACROS_DELAY:100;	

Example	CW_MACROS_DELAY;	
Note	Adjust the delay between the moment you've initiated TXing and when you actually TX.	

CW_KEYER_SPEED	Control CW keyer speed (WPM)	
Set	CW_KEYER_SPEED:arg1;	arg1 — CW speed, WPM (1...60).
Read	CW_KEYER_SPEED;	
Reply	CW_KEYER_SPEED:arg1;	
Type	Bidirectional control	
Example	CW_KEYER_SPEED:35; CW_KEYER_SPEED;	
Note	The server sends the current speed on connection and on every change.	

VOLUME	Main volume control	
Set	VOLUME:arg1;	arg1 — volume value, dB. The range of values is from -60 to 0 dB, at a value of -60 dB there is no sound.
Read	VOLUME;	
Reply	VOLUME:arg1;	
Type	Bidirectional control	
Example	VOLUME:-12; VOLUME;	

MUTE	Mute – disable/enable main volume	
Set	MUTE:arg1;	arg1 — status indicator (enable – true, disable – false).

Read	MUTE;	
Reply	MUTE:arg1;	
Type	Bidirectional control	
Example	MUTE:true; MUTE:false; MUTE;	

RX_MUTE	Mute a certain receiver	
Set	RX_MUTE:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	RX_MUTE:arg1;	
Reply	RX_MUTE:arg1,arg2;	
Type	Bidirectional control	
Example	RX_MUTE:0,true; RX_MUTE:1,false; RX_MUTE:0;	

RX_VOLUME	Volume control for each channel (VFO) in a software receiver	
Set	RX_VOLUME:arg1,arg2,arg3;	arg1 — receiver periodic number. arg2 — channel periodic number. arg3 — volume level, dB. The range of values is from -60 to 0 dB, at a value of -60 dB there is no sound.
Read	RX_VOLUME:arg1,arg2;	
Reply	RX_VOLUME:arg1,arg2,arg3;	
Type	Bidirectional control	
Example	RX_VOLUME:0,1,-6; RX_VOLUME:0,0;	

RX_BALANCE	Volume balance for each channel (VFO) in a software receiver	
Set	RX_BALANCE:arg1,arg2,arg3;	arg1 — receiver periodic number. arg2 — channel periodic number. arg3 — volume level, dB. Volume level, dB (-40...40). Negative values decrease volume in left channel, but the right channel stays the same. Positive values do the opposite.
Read	RX_BALANCE:arg1,arg2;	
Reply	RX_BALANCE:arg1,arg2,arg3;	
Type	Bidirectional control	
Example	RX_BALANCE:0,1,-6; RX_BALANCE:0,0,12; RX_BALANCE:0,1;	

MON_VOLUME	Control monitor volume in TX mode	
Set	MON_VOLUME:arg1;	arg1 — volume value, dB. The range of values is from -60 to 0 dB, at a value of -60 dB there is no sound.
Read	MON_VOLUME;	
Reply	MON_VOLUME:arg1;	
Type	Bidirectional control	
Example	MON_VOLUME:-12; MON_VOLUME;	

MON_ENABLE	Enable/disable monitoring in TX mode	
Set	MON_ENABLE:arg1;	arg1 — status indicator (enable – true, disable – false).
Read	MON_ENABLE;	
Reply	MON_ENABLE:arg1;	
Type	Bidirectional control	
Example	MON_ENABLE:true;	

	MON_ENABLE:false; MON_ENABLE;	

AGC_MODE	Control receiver AGC mode	
Set	AGC_MODE:arg1,arg2;	arg1 — receiver periodic number. arg2 — operation mode. List of supported modes: • normal; • fast; • off.
Read	AGC_MODE:arg1;	
Reply	AGC_MODE:arg1,arg2;	
Type	Bidirectional control	
Example	AGC_MODE:0,normal; AGC_MODE:1;	
Note	The mode is replied in lower case; the case of a set value does not matter. An unknown mode is ignored.	

AGC_GAIN	Control receiver AGC gain	
Set	AGC_GAIN:arg1,arg2;	arg1 — receiver periodic number. arg2 — gain value, dB. The range of values is from -20 to 120 dB.
Read	AGC_GAIN:arg1;	
Reply	AGC_GAIN:arg1,arg2;	
Type	Bidirectional control	
Example	AGC_GAIN:0,87; AGC_GAIN:1;	

RX_NB_ENABLE	Enable/disable receiver Noise Blanker (NB) which removes impulse interferences	
Set	RX_NB_ENABLE:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	RX_NB_ENABLE:arg1;	

Reply	RX_NB_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	RX_NB_ENABLE:0,true; RX_NB_ENABLE:1;	

RX_NB_PARAM	Adjust Noise Blanker (NB) parameters	
Set	RX_NB_PARAM:arg1,arg2,arg3;	arg1 — receiver periodic number. arg2 - triggering threshold, value range: 1 … 100. arg3 - pulse duration, value range: 1 … 300.
Read	RX_NB_PARAM:arg1;	
Reply	RX_NB_PARAM:arg1,arg2,arg3;	
Type	Bidirectional control	
Example	RX_NB_PARAM:0,70,25; RX_NB_PARAM:1;	
Note	Both parameters are mandatory in the Set form, otherwise the command is treated as a read request.	

RX_BIN_ENABLE	Enable/disable pseudo stereo (binaural - BIN)	
Set	RX_BIN_ENABLE:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	RX_BIN_ENABLE:arg1;	
Reply	RX_BIN_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	RX_BIN_ENABLE:0,true; RX_BIN_ENABLE:1;	
Note	Available in the AM, LSB, USB, CW and NFM modes only. In other modes the command is rejected and the server replies with the current state.	

RX_NR_ENABLE	Enable/disable Noise Reduction (NR) filter which removes constant noise from audio signal	
Set	RX_NR_ENABLE:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	RX_NR_ENABLE:arg1;	
Reply	RX_NR_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	RX_NR_ENABLE:0,true; RX_NR_ENABLE:1;	
Note	Available in the AM, LSB, USB, CW and NFM modes only. In other modes the command is rejected and the server replies with the current state.	

RX_ANC_ENABLE	Enable/disable Adaptive Noise Cancellation (ANC) processor which emphasizes voice signal over noise of the receiver	
Set	RX_ANC_ENABLE:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	RX_ANC_ENABLE:arg1;	
Reply	RX_ANC_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	RX_ANC_ENABLE:0,true; RX_ANC_ENABLE:1;	
Note	Available in the AM, LSB, USB and NFM modes only. In other modes the command is rejected and the server replies with the current state.	

RX_ANF_ENABLE	Enable/disable Automatic Notch Filter which removes tone signals from audio signal	
Set	RX_ANF_ENABLE:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	RX_ANF_ENABLE:arg1;	

Reply	RX_ANF_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	RX_ANF_ENABLE:0,true; RX_ANF_ENABLE:1;	
Note	Available in the AM, LSB, USB and NFM modes only. In other modes the command is rejected and the server replies with the current state.	

RX_APF_ENABLE	Enable/disable Analog Peak Filter which changes AFC that voice sounds more prominent over noises	
Set	RX_APF_ENABLE:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	RX_APF_ENABLE:arg1;	
Reply	RX_APF_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	RX_APF_ENABLE:0,true; RX_APF_ENABLE:1;	
Note	Available in the AM, LSB, USB, CW and NFM modes only. In other modes the command is rejected and the server replies with the current state.	

RX_DSE_ENABLE	Enable/disable Digital Surround Effect for CW signals, it provides space orientation of the CW signals in the filter bandwidth, depending on their position in the filter	
Set	RX_DSE_ENABLE:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	RX_DSE_ENABLE:arg1;	
Reply	RX_DSE_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	RX_DSE_ENABLE:0,true;	

	RX_DSE_ENABLE:1;	
Note	Available in the CW mode only. In other modes the command is rejected and the server replies with the current state.	

RX_NF_ENABLE	Enable/disable Notch Filters (NF) module	
Set	RX_NF_ENABLE:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	RX_NF_ENABLE:arg1;	
Reply	RX_NF_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	RX_NF_ENABLE:0,true; RX_NF_ENABLE:1;	

LOCK	Tuning frequency lock	
Set	LOCK:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	LOCK:arg1;	
Reply	LOCK:arg1,arg2;	
Type	Bidirectional control	
Example	LOCK:0,true; LOCK:1;	

SQL_ENABLE	Enable/disable squelch	
Set	SQL_ENABLE:arg1,arg2;	arg1 — receiver periodic number. arg2 — status indicator (enable – true, disable – false).
Read	SQL_ENABLE:arg1;	
Reply	SQL_ENABLE:arg1,arg2;	

Type	Bidirectional control	
Example	SQL_ENABLE:0,true; SQL_ENABLE:1;	
Note	Available in the AM, LSB, USB and NFM modes only. In other modes the command is rejected and the server replies with the current state.	

SQL_LEVEL	Adjust squelch threshold	
Set	SQL_LEVEL:arg1,arg2;	arg1 — receiver periodic number. arg2 — triggering threshold, dB. The range of values is from -140 to 0 dB.
Read	SQL_LEVEL:arg1;	
Reply	SQL_LEVEL:arg1,arg2;	
Type	Bidirectional control	
Example	SQL_LEVEL:0,-83; SQL_LEVEL:1;	

DIGL_OFFSET	Adjust the frequency offset for DIGL mode	
Set	DIGL_OFFSET:arg1;	arg1 — offset frequency, Hz. The range of values is from 0 Hz up to 4 kHz.
Read	DIGL_OFFSET;	
Reply	DIGL_OFFSET:arg1;	
Type	Bidirectional control	
Example	DIGL_OFFSET:1500; DIGL_OFFSET;	

DIGU_OFFSET	Adjust the frequency offset for DIGU mode	
Set	DIGU_OFFSET:arg1;	arg1 — offset frequency, Hz. The range of values is from 0 Hz up to 4 kHz.

Read	DIGU_OFFSET;	
Reply	DIGU_OFFSET:arg1;	
Type	Bidirectional control	
Example	DIGU_OFFSET:2200; DIGU_OFFSET;	

4.3 Unidirectional Control Commands

TX_ENABLE	Informs clients that TX is enabled or disabled for current transceiver	
Read	TX_ENABLE:arg1;	arg1 — transceiver periodic number;
Reply	TX_ENABLE:arg1,arg2;	arg2 — transmission allowed (true) / transmission denied (false).
Type	Bidirectional control	
Example	TX_ENABLE:0,true;	
Note	Sent to the client on connection and whenever the transmission permission changes (band, TX inhibit, receive antenna lock). Only ExpertSDR3 controls this parameter — a set request from a client is ignored, the server replies with the current state.	

CW_MACROS_SPEED_UP	CW macros speed increase (WPM)	
Set	CW_MACROS_SPEED_UP:arg1;	arg1 — the number of WPM by which to increase the CW speed.
Type	Unidirectional control	
Example	CW_MACROS_SPEED_UP:7; CW_MACROS_SPEED_UP:2;	
Note	Sent only by the TCI client.	

CW_MACROS_SPEED_	CW macros speed decrease (WPM)

DOWN		
Set	CW_MACROS_SPEED_DOWN:arg1;	arg1 — the number of WPM by which to decrease the CW speed.
Type	Unidirectional control	
Example	CW_MACROS_SPEED_DOWN:7; CW_MACROS_SPEED_DOWN:2;	
Note	Sent only by the TCI client.	

SPOT	Spot transfer for display on panorama	
Set	SPOT:arg1,arg2,arg3,arg4,arg5;	arg1 - callsign. arg2 - mode type. arg3 - frequency, Hz. arg4 - ARGB color. arg5 - additional text.
Type	Unidirectional control	
Example	SPOT:RN6LHF,CW,7100000,16711680,ANY_TEXT;	
Note	Sent only by the TCI client. The callsign, the frequency and the colour are mandatory, otherwise the command is ignored. The spot lifetime is 10 minutes.	

SPOT_DELETE	Delete spot	
Set	SPOT_DELETE:arg1;	arg1 — callsign.
Type	Unidirectional control	
Example	SPOT_DELETE:RN6LHF;	
Note	Sent only by the TCI client.	

IQ_SAMPLERATE	IQ stream sample rate	
Set	IQ_SAMPLERATE:arg1;	arg1 — sample rate, (Hz).

		Supported Sample rates: 48/96/192/384 kHz
Read	IQ_SAMPLERATE;	
Reply	IQ_SAMPLERATE:arg1;	
Type	Unidirectional control	
Example	IQ_SAMPLERATE:48000;	
Note	Sent only by the TCI client. The rate is individual for each client. An unsupported value is ignored, the server replies with the current rate.	

AUDIO_SAMPLERATE	LF audio stream sample rate	
Set	AUDIO_SAMPLERATE:arg1;	arg1 — sample rate, (Hz). Supported Sample rates: 8/12/24/48 kHz
Read	AUDIO_SAMPLERATE;	
Reply	AUDIO_SAMPLERATE:arg1;	
Type	Unidirectional control	
Example	AUDIO_SAMPLERATE:12000;	
Note	Sent only by the TCI client. The rate applies to the receiver and the transmitter audio streams of this client. An unsupported value is ignored, the server replies with the current rate.	

IQ_START	Start IQ stream	
Set	IQ_START:arg1;	arg1 — receiver periodic number.
Type	Unidirectional control	

Example	IQ_START:0;	
Note	Sent only by the TCI client.	

IQ_STOP	Stop IQ stream	
Set	IQ_STOP:arg1;	arg1 — receiver periodic number.
Type	Unidirectional control	
Example	IQ_STOP:0;	
Note	Sent only by the TCI client.	

AUDIO_START	Start LF audio stream	
Set	AUDIO_START:arg1;	arg1 — receiver periodic number.
Type	Unidirectional control	
Example	AUDIO_START:0;	
Note	Sent only by the TCI client.	

AUDIO_STOP	Stop LF audio stream	
Set	AUDIO_STOP:arg1;	arg1 — receiver periodic number.
Type	Unidirectional control	
Example	AUDIO_STOP:0;	
Note	Sent only by the TCI client.	

LINE_OUT_START	Start audio stream from Line Out	
Set	LINE_OUT_START:arg1;	arg1 — receiver periodic number.

Type	Unidirectional control	
Example	LINE_OUT_START:0;	
Note	Sent only by the TCI client.	

LINE_OUT_STOP	Stop audio stream from Line Out	
Set	LINE_OUT_STOP:arg1;	arg1 — receiver periodic number.
Type	Unidirectional control	
Example	LINE_OUT_STOP:0;	
Note	Sent only by the TCI client.	

LINE_OUT_RECORDER_START	Start recording of the audio stream from Line Out	
Set	LINE_OUT_RECORDER_START:arg1, arg2;	arg1 — receiver periodic number. arg2 — maximum recording time, s (1...300), mandatory.
Reply	LINE_OUT_RECORDER_START:arg1, arg2;	The server confirms the command with the applied duration.
Type	Unidirectional control	
Example	LINE_OUT_RECORDER_START:0,250;	
Note	Sent only by the TCI client. When the set recording time expires the recording is deleted and the server sends LINE_OUT_RECORDER_BREAK. To save the recording into a file, send the LINE_OUT_RECORDER_SAVE command.	

LINE_OUT_RECORDER_SAVE	Save audio stream recording from Line Out into a file	
Set	LINE_OUT_RECORDER_SAVE:arg1,arg2;	arg1 — receiver periodic number. arg2 — full name of the file.
Reply	LINE_OUT_RECORDER_SAVE:arg1,arg2;	
Type	Unidirectional control	
Example	LINE_OUT_RECORDER_SAVE:0,home/user_name/record_dir/file_name.wav; LINE_OUT_RECORDER_SAVE:0,home/user_name/record_dir/file_name.mp3; LINE_OUT_RECORDER_SAVE:0,D|\record_dir\file_name.mp3;	
Note	Sent only by the TCI client. Supported recording formats: WAVE and MP3. For Windows OS the path contains the forbidden character ":", it is replaced by "|", also you can use forward and backslash in the path to the file.	

LINE_OUT_RECORDER_BREAK	Stop audio stream recording from Line Out and delete the recording	
Set	LINE_OUT_RECORDER_BREAK:arg1;	arg1 — receiver periodic number.
Type	Unidirectional control	
Example	LINE_OUT_RECORDER_BREAK:0;	
Note	Sent only by the TCI client.	

SPOT_CLEAR	Delete all spots from panorama	
Set	SPOT_CLEAR;	
Type	Unidirectional control	
Example	SPOT_CLEAR;	
Note	Sent only by the TCI client.	

AUDIO_STREAM_SAMPLE_TYPE	Set sample format for audio stream	
Set	AUDIO_STREAM_SAMPLE_TYPE:arg1;	arg1 — sample format identifier Supported formats: • int16 • int24 • int32 • float32
Read	AUDIO_STREAM_SAMPLE_TYPE;	
Reply	AUDIO_STREAM_SAMPLE_TYPE:arg1;	The format is replied in lower case; the case of a set value does not matter.
Type	Unidirectional control	
Example	AUDIO_STREAM_SAMPLE_TYPE:int24; AUDIO_STREAM_SAMPLE_TYPE:float32;	
Note	An unknown format is ignored, the server replies with the current one. By default, sample format is float32.	

AUDIO_STREAM_CHANNELS	Set the number of channels for an audio stream	
Set	AUDIO_STREAM_CHANNELS:arg1;	arg1 — channels number

		Supported 1 or 2 channels.
Read	AUDIO_STREAM_CHANNELS;	
Reply	AUDIO_STREAM_CHANNELS:arg1;	
Type	Unidirectional control	
Example	AUDIO_STREAM_CHANNELS:1; AUDIO_STREAM_CHANNELS:2;	
Note	Values outside 1...2 are limited. By default, the number of channels equals 2.	

AUDIO_STREAM_SAMPLES	Set the amount of samples sent in a single packet	
Set	AUDIO_STREAM_SAMPLES:arg1;	arg1 — sample amount indicated in a *Stream.length* field. The value can vary from 100 up to 2048 samples.
Read	AUDIO_STREAM_SAMPLES;	
Reply	AUDIO_STREAM_SAMPLES:arg1;	
Type	Unidirectional control	
Example	AUDIO_STREAM_SAMPLES:200; AUDIO_STREAM_SAMPLES:512;	
Note	Values outside 100...2048 are limited. For every sample rate value there is a default sample amount (indicated in *Stream.length*): Sample rate 48 kHz - 2048 samples Sample rate 24 kHz - 1024 samples Sample rate 12 kHz - 512 samples Sample rate 8 kHz - 256 samples After you set the sample amount this value will be applied to every sample	

	rate. It is recommended to set the sample amount so that the length of playback duration is not less than 10 ms, otherwise the signal which is going to be transmitted may be corrupted. Recommended minimal sample amount: 48 kHz - 512 samples 24 kHz - 256 samples 12 kHz - 128 samples 8 kHz - 100 samples

TX_STREAM_AUDIO_BUFFERING	Set buffering timeout for TX signal	
Set	TX_STREAM_AUDIO_BUFFERING:arg1;	arg1 — buffering timeout, ms The range of values is from 50 up to 500.
Read	TX_STREAM_AUDIO_BUFFERING;	
Reply	TX_STREAM_AUDIO_BUFFERING:arg1;	
Type	Unidirectional control	
Example	TX_STREAM_AUDIO_BUFFERING:150; TX_STREAM_AUDIO_BUFFERING;	
Note	Default value is 50 ms. A larger value increases the delay but makes the transmitted signal more tolerant to network jitter.	

4.4 Notification Commands

CLICKED_ON_SPOT	Informs about click on the spot on the ExpertSDR3 panorama (Legacy command)	
Set	CLICKED_ON_SPOT:arg1,arg2;	arg1 — callsign. arg2 — frequency, Hz.

Type	Notification	
Example	CLICKED_ON_SPOT:RN6LHF,7147500;	
Note	Sent only by the TCI server. Sent for channel A only; the callsign is in upper case.	

RX_CLICKED_ON_SPOT	Informs about click on the spot on the ExpertSDR3 panorama	
Set	RX_CLICKED_ON_SPOT:arg1,arg2,arg3,arg4;	arg1 — receiver periodic number arg2 — channel periodic number (A/B). arg3 — callsign arg4 — frequency, Hz
Type	Notification	
Example	RX_CLICKED_ON_SPOT:0,1,RN6LHF,7147500;	
Note	Sent only by the TCI server.	

TX_FOOTSWITCH	Informs about PTT footswitch state	
Set	TX_FOOTSWITCH:arg1,arg2;	arg1 — transceiver periodic number arg2 — footswitch state (pressed - true, not pressed - false)
Type	Notification	
Example	TX_FOOTSWITCH:0,true;	
Note	Sent only by the TCI server.	

TX_FREQUENCY	Informs about current transmitter frequency	
Set	TX_FREQUENCY:arg1;	arg1 — transmission frequency, Hz
Type	Notification	

Example	TX_FREQUENCY:7140000;	
Note	Sent only by the TCI server.	

APP_FOCUS	Informs about the state of the ExpertSDR3 window (in focus or not)	
Set	APP_FOCUS:arg1;	arg1 — focus state (in focus - true, out of focus - false)
Type	Notification	
Example	APP_FOCUS:true;	
Note	Sent only by the TCI server.	

SET_IN_FOCUS	Make the main ExpertSDR3 window active (in focus)	
Set	SET_IN_FOCUS;	
Type	Unidirectional control	
Example	SET_IN_FOCUS;	
Note	Sent only by the TCI client.	

KEYER	Informs about CW Key state (not automatic)	
Set	KEYER:arg1,arg2,arg3;	arg1 – transceiver periodic number arg2 – CW key state (pressed -true, not pressed – false) arg3 – duration of the state this event ends, ms: for a press – the pause before it, for a release – how long the key was held
Type	Unidirectional control	
Example	KEYER:0,true,50;	
Note	Sent only by the TCI client. All three arguments are mandatory. The command lets a client work as a	

	CW key with the timing measured on its side: the device plays each state for the given duration if the next event arrives in time. Works in CW mode only. Example of a sequence: the first press – keyer:0,true,0; the release after 60 ms – keyer:0,false,60; the next press after a 60 ms pause – keyer:0,true,60;. This keeps the length of dots and dashes and minimizes the “drunken sailor” effect. The first press switches the transceiver to transmission. The transceiver returns to receive when the CW core of the device goes idle after the last release.

RX_SENSORS_ENABLE	Enable informing about signal level in RX filter bandwidth	
Set	RX_SENSORS_ENABLE:arg1,arg2;	arg1 – status indicator (enable – true, disable – false) arg2 – sending interval, ms (30-1000 ms, optional)
Type	Unidirectional control	
Example	RX_SENSORS_ENABLE:true; RX_SENSORS_ENABLE:true,200;	
Note	Sent only by the TCI client. Also enables the RX_CHANNEL_SENSORS notifications.	

TX_SENSORS_ENABLE	Enable informing about TX signal parameters	
Set	TX_SENSORS_ENABLE:arg1,arg2;	arg1 – status indicator (enable – true, disable – false) arg2 – sending interval, ms (30-1000 ms, optional)
Type	Unidirectional control	
Example	TX_SENSORS_ENABLE:true; TX_SENSORS_ENABLE:true,200;	
Note	Sent only by the TCI client.	

RX_SENSORS	Informs about signal level in RX filter bandwidth	
Set	RX_SENSORS:arg1,arg2;	arg1 – receiver periodic number arg2 – signal level, dBm
Type	Notification	

Example	RX_SENSORS:0,-71.5; RX_SENSORS:1,-112.7;	
Note	Sent only by the TCI server. Sent only for channel A and only when the level changes.	

TX_SENSORS	Informs about TX signal parameters	
Set	TX_SENSORS:arg1,arg2,arg3,arg4,arg5;	arg1 - transceiver periodic number arg2 - microphone signal level, dBm arg3 - signal power at the transmitter output, W (RMS) arg4 - peak signal power at the transmitter output, W arg5 - SWR
Type	Notification	
Example	TX_SENSORS:0,-27.2,47.4,67.5,1.7;	
Note	Sent only by the TCI server.	

VFO_LOCK	Notification about locking the frequency tuning by the client	
Reply	VFO_LOCK:arg1,arg2,arg3;	arg1 — receiver periodic number; arg2 — channel periodic number (A/B); arg3 — lock status (true/false).
Type	Notification	
Example	VFO_LOCK:0,1,true; VFO_LOCK:1,1,false;	
Note	Sent only by the TCI server. A client can neither set nor read it.	

RX_CHANNEL_SENSORS	Signal level readings in RX bandwidth per channel	
Set	RX_CHANNEL_SENSORS:arg1,arg2,arg3;	arg1 — receiver periodic number arg2 — channel periodic number (A/B) arg3 — signal level, dBm

Type	Notification	
Example	RX_CHANNEL_SENSORS:0,0,-71.5; RX_CHANNEL_SENSORS:1,1,-112.7;	
Note	Sent only by the TCI server.	

4.5 New Commands in this version

RX_ENABLE	Enable software transceiver.	
Set	RX_ENABLE:arg1,arg2;	arg1 — transceiver periodic number; arg2 — status indicator (enable – true, disable – false).
Read	RX_ENABLE:arg1;	
Reply	RX_ENABLE:arg1,arg2;	
Type	Bidirectional control	
Example	RX_ENABLE:1,true; RX_ENABLE:1;	
Note	The first transceiver is always enabled. A disabled transceiver rejects TRX and TUNE commands.	
ECODER_SWITCH_RX	Active receiver of the E-Coder panel.	
Set	ECODER_SWITCH_RX:arg1,arg2;	arg1 — panel periodic number (0–19); arg2 — receiver periodic number.
Read	ECODER_SWITCH_RX:arg1;	
Reply	ECODER_SWITCH_RX:arg1,arg2;	
Type	Bidirectional control	
Example	ECODER_SWITCH_RX:0,1; ECODER_SWITCH_RX:0;	
Note	When the number of connected panels changes, the server sends the state of each panel.	
ECODER_SWITCH_CHANNEL	Active channel (A/B) of the E-Coder panel.	

Set	ECODER_SWITCH_CHANNEL:arg1,arg2;	arg1 — panel periodic number (0–19); arg2 — channel number (A/B).
Read	ECODER_SWITCH_CHANNEL:arg1;	
Reply	ECODER_SWITCH_CHANNEL:arg1,arg2;	
Type	Bidirectional control	
Example	ECODER_SWITCH_CHANNEL:0,1; ECODER_SWITCH_CHANNEL:0;	
HW_SENSORS_ENABLE	Enable notifications from hardware sensors.	
Set	HW_SENSORS_ENABLE:arg1,arg2;	arg1 — status indicator (enable – true, disable – false); arg2 — notification interval, ms (30–1000), optional.
Type	Unidirectional control	
Example	HW_SENSORS_ENABLE:true,500;	
Note	Enables the HW_SENSORS notification.	
CW_MACROS	Transmit text in CW.	
Set	CW_MACROS:arg1,arg2;	arg1 — transceiver periodic number; arg2 — text.
Type	Unidirectional control	
Example	CW_MACROS:0,TU RA6LH 599;	
Note	For the text syntax, see the «Working in CW mode» section.	
CW_MSG	Transmit a CW message with a callsign.	
Set	CW_MSG:arg1,arg2,arg3,arg4;	arg1 — transceiver periodic number; arg2 — prefix; arg3 — callsign; arg4 — suffix.
Set	CW_MSG:arg1;	arg1 — corrected callsign.
Type	Unidirectional control	
Example	CW_MSG:0,TU,RA6LH,599 004;	

	CW_MSG:RA6LH;	
Note	The single-argument form corrects the callsign that has not been sent to the air yet.	
CW_MACROS_STOP	Immediate stop of CW transmission.	
Set	CW_MACROS_STOP;	
Type	Unidirectional control	
Example	CW_MACROS_STOP;	
Note	Sent only by the TCI client.	
CW_TERMINAL	Enable the CW terminal mode.	
Set	CW_TERMINAL:arg1,arg2;	arg1 — transceiver periodic number; arg2 — status indicator (enable – true, disable – false).
Reply	CW_TERMINAL:arg1,arg2;	
Type	Unidirectional control	
Example	CW_TERMINAL:0,true;	
Note	Both arguments are mandatory. In terminal mode the transceiver stays in TX after the macro has been sent.	
HW_SENSORS	Hardware sensors of the device.	
Reply	HW_SENSORS:arg1,arg2,arg3;	arg1 — supply voltage, V; arg2 — current consumption, A; arg3 — temperature, ℃.
Type	Notification	
Example	HW_SENSORS:13.8,4.5,42.0;	
Note	Enabled by the HW_SENSORS_ENABLE command. Sent only by the TCI server.	
CW_MON	A character has been sent in CW.	
Reply	CW_MON:arg1;	arg1 — the sent character.
Type	Notification	

Example	CW_MON:T;	
Note	Allows the client to display the CW transmission in a terminal window synchronously with the air. Sent only by the TCI server. The command works since ExpertSDR3 1.2.0; it was first described in the TCI 2.1 specification.	
CALLSIGN_SEND	The callsign of the CW message has been sent.	
Reply	CALLSIGN_SEND:arg1;	arg1 — the sent callsign.
Type	Notification	
Example	CALLSIGN_SEND:ra6lh;	
Note	Contains the final version of the callsign, including corrections made by the CW_MSG command, in lower case. Sent only by the TCI server.	
CW_MACROS_EMPTY	The CW macro queue is empty.	
Reply	CW_MACROS_EMPTY:arg1;	arg1 — transceiver periodic number.
Type	Notification	
Example	CW_MACROS_EMPTY:0;	
Note	Sent at the moment the last character of the queue starts transmitting. Sent only by the TCI server.	

4.6 Reserved Commands

The commands below are part of the protocol dictionary, but the TCI server does not accept or send them, so a client must not rely on them.

Command	Purpose
PREAMP, PREAMP_PARAM	Preamp control.
CTCSS_ENABLE, CTCSS_MODE, CTCSS_RX_TONE, CTCSS_TX_TONE, CTCSS_LEVEL	CTCSS control in NFM mode.
RX_NF_PARAM	Parameters of a separate notch filter. The enable flag is available as RX_NF_ENABLE.
RX_SMETER, TX_POWER, TX_SWR	Meters of TCI 1.x versions, replaced with RX_SENSORS and TX_SENSORS.

4.7 Notes on the Command Format

TCI server behavior that is not visible from the command tables.

1. Empty arguments are discarded during parsing. Therefore VFO:0,,7100000; is parsed as the two-argument VFO:0,7100000;, that is, as a read request of channel 7100000, which does not exist, and is ignored. Empty arguments must not be sent.
2. A read request differs from a set request by a missing argument. If an argument cannot be parsed in a set request — for example, a negative frequency in an unsigned field — the whole argument is considered missing, and the command turns into a read request.
3. A logical argument is true if it equals true or 1 in any case; any other value is considered false.
4. Frequencies and levels are returned as integers, for example RIT_OFFSET:0,500;. The fractional part is used only where the value itself is fractional — voltage and current in HW_SENSORS.
5. Ignored without a reply: an unknown command, a command with an argument of a wrong type, and a command with a receiver number out of range.
6. A parameter changed by ExpertSDR3 or another client is monopolized for 200 ms (CW speed and delay — for 500 ms). A set request arriving within this window is dropped, and for commands that always reply, the server returns the current value.
7. The state of a parameter is sent to clients not immediately, but on a 30 ms tick. Several fast changes of one parameter are merged into a single notification.

Appendix A. Glossary of Technical Terms

Terminology	Description